

SEMANTIC KERNEL PROMPTING

A Practitioner's Guide to Structured Reading Through Dialogue

Multi-Pass Question-Driven Analysis for Complex Regulatory and Technical Texts

CoAgnes Framework

Working Document — March 2026

Document ID: SKP-GUIDE-2026-001

Abstract

This document specifies a multi-pass prompting protocol—termed Semantic Kernel Prompting—for the structured analysis of complex regulatory and technical texts using large language model (LLM) chat interfaces. The protocol decomposes the act of reading into a sequence of discrete operations: question spectrum generation, oriented context assembly, disclosure through an attention geometry, crystallization of structural fragments, and trace-based verification. Each operation has formally defined inputs, outputs, and dependency relations to the preceding steps. The protocol is demonstrated on GDPR regulatory fragments and is presented in a form suitable for immediate manual application within standard chat interfaces (ChatGPT, Claude), with extensions toward workflow automation discussed as a natural generalization.

The central contribution is the treatment of questions as primary structural objects that persist across execution steps, functioning not as retrieval queries but as semantic registers that shape the field of attention within which analysis occurs. This shifts the analytical paradigm from answer extraction to field formation—producing outputs that are not summaries but structured semantic maps with explicit completeness and uncertainty markers.

1. Service: Output Characterization

1.1 Problem Statement

Single-pass LLM interaction with complex texts produces outputs that exhibit a characteristic pathology: apparent completeness masking structural incompleteness. When a practitioner submits a regulatory fragment with the instruction “summarize this,” the model generates a fluent response that covers the most salient features of the text while silently omitting boundary conditions, unresolved ambiguities, implicit dependencies, and the fragment’s relational position within its broader normative environment. The output appears sufficient; its insufficiency is invisible without independent analysis.

This pathology is particularly acute in regulatory domains where texts function as nodes in dense referential networks. A GDPR recital, for example, derives its operative meaning not from its surface propositions alone but from its connections to defined terms (Article 4), operative principles (Article 5), specific obligations (Articles 24, 25, 32), and interpretive boundaries established by adjacent recitals. A flat summary severs these connections, producing an artifact that resembles understanding while lacking its structural foundations.

1.2 Protocol Output Structure

The Semantic Kernel Protocol produces a fundamentally different class of output: a multi-layered semantic map of the analyzed fragment. This map is organized along seven projective axes, each serving a distinct analytical function:

Axis	Function	Output Type
Truth	Anchors the fragment’s definitional ground	Critical definitions, term resolutions, reference linkages
Deep	Delineates applicability boundaries	Boundary questions, scope conditions, modality determinations
Connect	Positions the fragment in its relational field	Structural connections to adjacent norms, principles, and mechanisms
Service	Crystallizes operative semantic forms	Condensed formulas, if/then structures, negation frames, practical models
Knowledge	Enumerates the fragment’s referential neighborhood	Lists of related provisions with typed relationships
Evolution	Maps temporal and causal dependencies	Sequencing of states, regulatory genealogy, maintenance obligations
Responsibility	Marks points of genuine incompleteness	Open interpretive questions, under-determined boundaries, delegated specifications

The Responsibility axis is of particular methodological significance. By explicitly marking what the fragment does not resolve—rather than allowing such gaps to remain invisible—the protocol converts omission from a latent defect into an explicit structural element of the output. The practitioner receives not only what is known but a map of what remains genuinely open.

1.3 Differentiation from Conventional Approaches

The protocol's output differs from standard summarization in three respects. First, it preserves referential structure: cross-references, definitional dependencies, and relational positions are explicit rather than dissolved. Second, it maintains boundary information: the conditions under which the fragment applies, fails to apply, or applies with qualification are surfaced as first-class elements. Third, it carries incompleteness markers: points where the text is genuinely ambiguous, delegated, or under-specified are identified and catalogued rather than silently resolved through the model's interpolation.

2. Connect: Relation to Established Methods

2.1 Chain-of-Thought Decomposition

The protocol shares with chain-of-thought (CoT) prompting the principle that reasoning quality improves when intermediate steps are made explicit. However, where CoT operates within a single generation—decomposing one inference into visible sub-steps—the Semantic Kernel Protocol distributes decomposition across multiple passes, each with a distinct cognitive function. The question-generation pass (Step 2) operates in exploratory mode; the context-assembly pass (Step 3) operates in projective mode; the disclosure pass (Step 4) operates in analytical mode. This functional separation prevents the characteristic CoT failure mode in which exploratory and analytical reasoning contaminate one another within a single generation.

2.2 Multi-Perspective Analysis

The triaxial question structure (Truth/Deep/Connect) instantiates a specific form of multi-perspective analysis comparable to established frameworks such as de Bono’s lateral thinking hats or PESTLE environmental scanning. The distinctive property of the present protocol is that its axes are not arbitrary analytical lenses but functionally differentiated roles within the execution cycle: Truth questions serve as retrieval anchors for context assembly, Deep questions serve as attention constraints during disclosure, and Connect questions serve as projection coordinates for output structuring. The axes are not interchangeable perspectives applied to the same operation; they are distinct operations performed at distinct phases.

2.3 Retrieval-Augmented Generation

Step 3 of the protocol (context assembly) shares structural similarity with the retrieval-augmented generation (RAG) pattern. Both assemble external information into the model’s working context before generation. The critical divergence is in the retrieval mechanism: where RAG systems typically employ embedding-based similarity search over pre-chunked documents, the protocol uses semantically oriented questions as retrieval anchors. The questions encode not merely topical relevance but directional intent—a Truth question retrieves definitional grounding, a Deep question retrieves boundary information, a Connect question retrieves relational structure. This orientation produces contextual assemblies that are structured by analytical function rather than by embedding proximity.

2.4 Prompt Chaining and Multi-Step Workflows

At the operational level, the protocol is a manually executed prompt chain with defined handoff points between steps. Each step consumes the output of its predecessor and produces an input for its successor. This places the protocol in the broader family of multi-step LLM workflows, with the specific contribution being the formal definition of step semantics, inter-step dependencies, and the explicit role assignment of questions as persistent structural objects that span the entire chain.

2.5 Distinctive Synthesis

The protocol's novelty lies not in any individual technique but in the specific combination of three properties: (a) questions function as primary structural objects persisting across all execution steps, not as ephemeral retrieval queries; (b) context assembly is formally separated from analytical disclosure, preventing the model from analyzing material it has not yet properly contextualized; and (c) incompleteness is tracked explicitly as a first-class output dimension. These three properties together produce a qualitatively different output class than any single technique achieves in isolation.

3. Deep: Internal Coherence and Boundary Conditions

3.1 Step Dependency Structure

The protocol's steps are not independently applicable techniques but a dependency chain in which each step's output constitutes a necessary precondition for the next. This dependency structure is formally characterized as follows:

Step	Depends On	Produces	Consumed By
0. Input Declaration	—	Fragment metadata + text	Step 1
1. Orientation Loading	Step 0 (fragment type)	Question-generation prompt	Step 2
2. Question Generation	Step 1 + fragment text	Triaxial question spectra	Steps 3, 4
3. Context Assembly	Step 2 (Truth questions) + fragment	Seven-axis information projection	Step 4
4. Disclosure	Step 3 (context) + Step 2 (Deep, Connect)	Structured analytical output	Step 5
5. Fragment Extraction	Step 4 (disclosure output)	Typed semantic fragments (YAML)	Steps 6–7
6. Trace Recognition	Steps 5, 4, 3, 2	Root-to-fragment provenance chains	Step 7
7. Completeness Mapping	Steps 6, 5, 2	Question-to-fragment coverage map	—

Violating the dependency order produces predictable degradation. Omitting Step 2 (question generation) and proceeding directly to analysis eliminates the attention geometry, causing the model to distribute analytical effort according to surface salience rather than structural need. Omitting Step 3 (context assembly) forces the disclosure step to operate without oriented grounding, increasing the probability of definitional drift and referential error. Merging Steps 3 and 4 into a single pass combines context formation and analysis within one generation, reintroducing the contamination problem that CoT decomposition seeks to avoid.

3.2 Functional Role Differentiation of Question Axes

The three question axes (Truth, Deep, Connect) are not categories into which questions are sorted post hoc. They are functional specifications that determine how questions operate within the execution cycle:

- **Truth questions** operate at Step 3: they serve as retrieval anchors for context assembly, directing the model to project definitional and referential information into the working context. Their function is ground preparation.
- **Deep questions** operate at Step 4 as attention geometry: they constrain what the model attends to during disclosure, focusing analytical effort on boundary conditions, modality, and scope. Their function is field shaping.
- **Connect questions** operate at Step 4 as projection geometry: they determine the structural coordinates along which the disclosure output is organized. Their function is output formation.

This functional differentiation means that the same question, if moved from one axis to another, would perform a different role in the execution cycle. A boundary question placed in the Truth axis would be used to load boundary information into context but would not shape the attention geometry of disclosure. The axes are operational positions, not taxonomic labels.

3.3 Commitment Points

The protocol contains two deliberate reduction points—moments where the practitioner makes an explicit commitment that constrains subsequent execution:

First commitment (between Steps 2 and 3): The practitioner reviews the generated question spectra and decides which questions will guide the reading. This is an act of editorial judgment: adding missing questions, removing irrelevant ones, and accepting the resulting set as the operative field of attention. Once committed, these questions shape all subsequent steps.

Second commitment (between Steps 4 and 5): The practitioner reviews the disclosure output and decides which structural fragments to extract and type. This is an act of crystallization: selecting which semantic units will persist as discrete, reusable objects. Fragments not extracted remain part of the disclosure prose but do not enter the structured artifact inventory.

These commitment points are not optional conveniences. They are structural features that prevent the protocol from collapsing into a single undifferentiated pass. Each commitment introduces a moment of human judgment that the model cannot perform on its own behalf—a deliberate asymmetry between human authority over trajectory and model authority over execution.

3.4 Protocol Boundaries

The protocol's internal boundaries define what it governs and what it does not:

- **Governed:** The sequence and dependency of analytical steps; the functional role of question axes; the separation of context assembly from disclosure; the structure of output artifacts; the requirement for incompleteness marking.
- **Not governed:** The specific content of orientation prompts (these are domain-adaptable); the number of questions generated per axis (this varies by fragment complexity); the criteria for fragment extraction (these depend on the practitioner's analytical purpose); the depth of trace verification (this ranges from informal review to systematic audit).

The protocol is therefore a structural specification of how analysis proceeds, not a content specification of what analysis discovers. It defines the geometry of the analytical act while leaving the substance to the interaction between practitioner, fragment, and model.

4. Truth: Stepwise Protocol Specification

This section constitutes the formal algorithmic description of the protocol. Each step is specified by its preconditions, inputs, operation, outputs, and invariants.

4.1 Step 0: Input Declaration

Precondition: A text fragment has been selected for analysis.

Operation: The practitioner explicitly declares the fragment's metadata as structured input parameters:

Parameter	Type	Description
epi_type	Enumerated string	Information domain (e.g., legal_regulation, technical_specification, news)
source_name	String	Source document identifier (e.g., GDPR, Basel III, RFC 9110)
fragment_type	String	Structural type within source (e.g., recital, article, section, clause)
fragment_name	String	Unique fragment identifier (e.g., GDPR Recital 29)
fragment_text	String	Full text of the fragment

Output: A typed, identified fragment ready for processing.

Invariant: No analytical operation may commence before input declaration. The metadata anchors the model in a specific semantic position within a specific document and domain, preventing generic or decontextualized analysis.

4.2 Step 1: Orientation Prompt Loading

Precondition: Step 0 is complete; fragment metadata is available.

Operation: Based on the combination of `epi_type`, `source_name`, and `fragment_type`, the practitioner selects or constructs an orientation prompt. This prompt is a question-generation instruction organized along three semantic axes:

- **Truth axis:** Instructions to generate questions pointing toward definitions, term resolutions, and referential anchors required for semantic grounding.
- **Deep axis:** Instructions to generate questions pointing toward the fragment's boundary conditions, scope limitations, modality, and applicability constraints.
- **Connect axis:** Instructions to generate questions pointing toward the fragment's position within the broader semantic field of its source document or domain.

The orientation prompt does not instruct the model to analyze the fragment. It instructs the model to generate the questions through which analysis will subsequently proceed. This distinction is essential: the prompt shapes the field of inquiry, not the content of analysis.

Output: A question-generation prompt tailored to the fragment's type and domain.

Invariant: The orientation prompt must address all three axes. Omitting an axis eliminates a functional role from the execution cycle (ground preparation, field shaping, or output formation, respectively).

4.3 Step 2: Question Spectrum Generation

Precondition: Orientation prompt (Step 1) and fragment text (Step 0) are available.

Operation: The orientation prompt is placed in the developer/system message. The fragment name and text are placed in the user message. The model is invoked. The output is a structured set of questions organized by axis.

Prompt assembly:

- **Developer message:** Orientation prompt (triaxial question-generation instruction).
- **User message:** Fragment name + fragment text.

Output: Three question sets (Truth, Deep, Connect), each containing questions with optional reference annotations (e.g., pointers to specific articles or definitions).

Post-generation review: The practitioner reviews the generated questions before proceeding. This review constitutes the first commitment point. The practitioner may: add questions that the model omitted; remove questions that are irrelevant to the analytical purpose; reformulate questions for greater precision; accept the question set as the operative field of attention.

Invariant: The question spectra must be finalized before any subsequent step executes. Questions are not generated incrementally during analysis; they are established as the complete field of inquiry prior to context assembly.

4.4 Step 3: Context Assembly

Precondition: Finalized question spectra (Step 2) and fragment text (Step 0) are available.

Operation: A context-projection prompt is constructed. This prompt instructs the model to use the Truth questions from Step 2 as anchoring vectors and to project a multi-dimensional information map covering seven sections:

Section	Projection Instruction
Truth	Critical definitions required for semantic completeness of the fragment
Deep	Questions for determining applicability and scope
Connect	Structural interrelationships with surrounding norms or components
Service	Concrete operative semantic forms (formulas, models, examples)
Knowledge	Enumeration of related fragments and provisions
Evolution	Temporal definitions, causal sequences, maintenance obligations
Responsibility	Points of genuine incompleteness, delegated specifications, interpretive gaps

Prompt assembly:

- **Developer message:** Context-projection instruction with seven-section structure, followed by the Truth questions from Step 2.
- **User message:** Fragment name + fragment text.

Output: A structured information projection organized along seven axes, grounded in the fragment's definitional dependencies and referential context.

Rationale for separation: This step is formally separated from disclosure (Step 4) because it serves a distinct function: loading the model's working memory with oriented contextual information before analytical operations commence. This separation is analogous to the distinction in legal analysis between reading the relevant definitions, cross-references, and interpretive guidance (context assembly) and performing the substantive analysis of a provision's meaning and implications (disclosure). Merging these operations within a single prompt degrades both: the model begins analyzing before it has fully contextualized, producing analysis grounded in incomplete or misoriented context.

4.5 Step 4: Disclosure Execution

Precondition: Context projection (Step 3) and question spectra (Step 2, specifically Deep and Connect sets) are available.

Operation: A disclosure prompt is constructed that combines three elements with distinct functional roles:

- **Attention geometry (developer message):** The Deep questions from Step 2, framing what the model should attend to during analysis—boundary conditions, modality, scope, applicability.
- **Projection geometry (developer message):** The Connect questions from Step 2 reformulated as a seven-axis output structure (Truth, Deep, Connect, Service, Knowledge, Evolution, Responsibility), defining how the analytical output should be organized.
- **Assembled context + fragment (user message):** The full context projection from Step 3, concatenated with the original fragment text.

Prompt assembly:

- **Developer message:** Disclosure instruction + attention geometry (Deep questions) + projection geometry (seven-axis output format derived from Connect questions).
- **User message:** Context projection output (Step 3) + fragment text (Step 0).

Output: A structured analytical disclosure of the fragment, organized along seven axes, informed by the assembled context, shaped by the attention geometry, and projected through the output geometry.

Invariant: The disclosure step operates within the context assembled at Step 3 and the question field established at Step 2. This constraint ensures that the analytical output is traceable to its informational sources and that no ungrounded interpolation enters the result without being identifiable as such.

4.6 Step 5: Structural Fragment Extraction

Precondition: Disclosure output (Step 4) is available.

Operation: The practitioner instructs the model to extract discrete semantic units from the disclosure output. Each unit is a typed structural fragment described by a fixed schema:

Field	Description
Name	A concise identifier for the fragment
Anchor	A brief semantic summary (one sentence)
Core Question	The question lying at the foundation of this fragment, in wh-fronted predicate-argument form
Sustaining Forces	Semantic conditions that make this fragment valid (textual, systemic, functional, logical)
Internal Connections	References to other fragments within the same disclosure output
Dependencies	References to fragments on which this one structurally depends
Full Text	The complete formulation of the fragment

Fragment types include, but are not limited to: assertions (what the text claims), definitions (what terms mean in context), constraints (what the text limits or prohibits), connections (how the fragment relates to its environment), and open questions (what the text does not resolve).

Output: A set of typed, structured semantic fragments with explicit provenance, dependencies, and sustaining conditions.

Second commitment point: The practitioner decides which fragments to extract and how to type them. This is an editorial act that determines the composition of the persistent output artifact.

4.7 Steps 6–7: Trace Recognition and Completeness Verification

Precondition: Extracted fragments (Step 5), disclosure output (Step 4), context projection (Step 3), and question spectra (Step 2) are all available.

Step 6 — Trace Recognition: For each extracted fragment, the model identifies its roots across three levels: (a) the specific questions from Step 2 that ground its existence; (b) the specific passages in the Step 4 disclosure from which it was crystallized; and (c) the specific context elements from Step 3 that provided its informational foundation. The result is a provenance chain linking each fragment back through the execution cycle to its originating questions and assembled context.

Step 7 — Completeness Verification: For each question posed in Step 2 as part of the disclosure geometry, the model maps which extracted fragments address it and to what degree. The result is a coverage matrix: questions fully addressed, partially addressed, or not addressed by the extracted fragment set. Unaddressed questions indicate either analytical gaps (the disclosure covered the question but no fragment was extracted) or genuine incompleteness (the disclosure could not adequately resolve the question given the available context).

Output: A provenance map (fragment → source chain) and a coverage map (question → fragment coverage). Together, these constitute the trace of the analytical pass.

Invariant: No fragment may exist without a traceable root in the question spectra and context projection. Questions without answers must be clearly identified. These statements ensure that the output is both grounded (nothing appears from nowhere) and complete (nothing is silently omitted).

5. Knowledge: Working with Protocol Outputs

5.1 Fragment Taxonomy

The protocol produces several classes of persistent semantic fragments, each serving a distinct function in downstream analytical work:

- **Assertions** encode what the fragment positively claims. They carry sustaining forces (the conditions under which the assertion holds), dependencies (other assertions or definitions required for validity), and explicit scope markers. Assertions are the primary carriers of substantive meaning.
- **Definitions** anchor the terminological ground. They resolve terms to their operative meanings within the source document, cross-referencing formal definitions where available. Definitions from earlier analyses become pre-loaded context for subsequent fragment analyses within the same document.
- **Boundaries** delineate where the fragment's applicability starts and stops. They record conditions of inclusion (when the fragment applies), conditions of exclusion (when it does not), and zones of ambiguity (where applicability is under-determined).
- **Connections** record typed relationships between the analyzed fragment and other elements in its normative or conceptual environment. Connection types include: depends-on, supports, constrains, specializes, generalizes, conflicts-with, and co-occurs-with.
- **Open Questions** record points where the fragment is genuinely incomplete, ambiguous, or delegated. These are not analytical failures but positive findings—they mark the frontier of what the text itself determines.

5.2 Versioning Through Conversation Branching

Modern chat interfaces (ChatGPT, Claude) natively support conversation branching—the ability to create divergent continuations from any point in a dialogue. This feature provides a natural mechanism for managing analytical versions and perspectives within the protocol:

- **Main thread:** Sequential analysis of fragments in document order. Each analysis enriches the cumulative context available for subsequent fragments.
- **Analytical branches:** From any point in the main thread, a branch may be created to explore a specific assertion in greater depth, test an alternative interpretation, or analyze a cross-referenced fragment before returning to the sequential flow.
- **Perspective branches:** Multiple branches from the same point may explore the same fragment through different orientation prompts or question sets, producing parallel analytical views that can be compared or synthesized.

This branching model is limited by the chat interface’s context window: branches share the parent context up to the divergence point but accumulate independently thereafter. For extended analyses, this necessitates explicit context management—summarizing or excerpting key findings from one branch before initiating another.

5.3 Cumulative Context Construction

When the protocol is applied sequentially to multiple fragments from the same document, each analysis enriches the available context for subsequent analyses. Definitions extracted from earlier fragments become part of the Truth context for later ones. Connections identified in earlier analyses extend the Connect field for later fragments. Open questions from earlier analyses may be resolved—or sharpened—by later analyses.

This cumulative property means that the protocol’s value increases nonlinearly with the number of fragments analyzed within a single document. The twentieth fragment analysis operates within a richer contextual field than the first, producing denser connections and more precise boundary determinations.

5.4 Fragment Portability

Extracted fragments are self-contained semantic units with explicit dependencies and sustaining conditions. This makes them portable: a fragment extracted from one conversation can be introduced into a new conversation as pre-loaded context, providing a practitioner with a persistent, reusable semantic toolkit for a domain. In practical terms, this means maintaining a document or notebook of key extracted fragments (definitions, boundary conditions, structural connections) that can be pasted into new analytical sessions as grounding material.

6. Evolution: Toward Automation

6.1 Protocol as Workflow Graph

Each step of the protocol is a discrete operation with formally defined inputs, a prompt template, and a structured output. This property makes the protocol naturally amenable to representation as a workflow graph in automation platforms such as N8N, where each node corresponds to a protocol step and edges represent data flow between steps.

6.2 Node Mapping

Protocol Step	N8N Node Type	Input	Output
0. Input Declaration	Trigger / Form	Fragment metadata + text	Typed fragment object
1. Orientation Loading	Switch / Lookup	Fragment type	Prompt template
2. Question Generation	LLM Call	Prompt template + fragment	Triaxial question set
3. Context Assembly	LLM Call	Truth questions + fragment	Seven-axis context
4. Disclosure	LLM Call	Context + Deep/Connect questions	Structured disclosure
5. Fragment Extraction	LLM Call + Parser	Disclosure output	Typed fragments (YAML/JSON)
6–7. Trace + Verify	LLM Call + Validator	All prior outputs	Provenance + coverage maps (YAML/JSON)

6.3 Capabilities Added by Automation

Workflow automation introduces capabilities that are structurally unavailable in manual chat execution:

- **Persistent storage:** Extracted fragments are written to a database (e.g., Qdrant, PostgreSQL) and indexed for retrieval, enabling cross-session continuity and cumulative knowledge graph construction.
- **Real retrieval:** Step 3 can integrate with vector databases or document stores to retrieve actual cross-references and definitions rather than relying on the model's parametric knowledge. For documents like GDPR this is a quality improvement; for proprietary or niche documents it is a prerequisite for reliable analysis.
- **Automated tracing:** Steps 6–7 can be executed systematically for every fragment, producing a complete provenance and coverage audit that is prohibitively labor-intensive in manual mode.
- **Document-scale execution:** The protocol can be applied to every fragment in a document sequentially, with outputs from each fragment automatically enriching the context for subsequent fragments, producing a complete semantic map of an entire regulatory instrument.

The transition from manual chat execution to automated workflow is not a replacement but a generalization: the practitioner's manual experience with the protocol informs the design of prompt templates, output parsers, and quality thresholds that govern the automated pipeline.

7. Responsibility: Limitations and Horizons

7.1 Structural Limitations of Chat-Mode Execution

The chat-mode adaptation of the protocol is immediately usable and produces valuable analytical artifacts. However, it operates under structural constraints inherent to the chat interface that limit its reliability and scalability:

- **No persistent state.** Each conversation begins from an empty context. The cumulative semantic field built through sequential fragment analysis does not survive between sessions. Continuity requires manual re-injection of key findings.
- **No real retrieval.** Step 3 (context assembly) relies entirely on the model's parametric knowledge. For well-documented public instruments (GDPR, Basel III), this is often adequate. For proprietary, niche, or recently published documents, the model's knowledge may be incomplete, outdated, or absent, producing context projections grounded in interpolation rather than source material.
- **No automated tracing.** Steps 6–7 require manual effort that is cognitively expensive and rarely performed thoroughly. In practice, most chat-mode analyses omit formal verification, relying on the practitioner's informal review of the disclosure output.
- **Context window degradation.** During the sequential analysis of multiple fragments, information accumulates in the LLM's chat context window until earlier material begins to silently fade from the model's active focus or becomes distorted as a result of semantic drift. The practitioner has limited visibility into what has been lost.
- **No structural ethical governance.** The protocol in chat mode relies on the model's built-in alignment and the practitioner's judgment. There is no mechanism for explicit ethical checkpoints, constraint inheritance, or governed trajectory modification.

7.2 What the Full Protocol Implies

The limitations enumerated above are not incidental inconveniences but indicators of the gap between the chat-mode adaptation and the protocol's full architectural potential. The complete protocol, when implemented at the infrastructure level, implies a **stepwise semantic execution runtime** in which:

- Each step is a traced, governed state transition with full provenance recording.
- Context is assembled dynamically from persistent knowledge stores, not from parametric memory.
- Extracted fragments are first-class persistent objects with typed relationships, versioning, and dependency tracking.
- Ethical governance is a structural invariant present at every state transition, not an optional post-hoc review.
- The entire analytical pass is auditable, replayable, and available as training signal for meta-learning systems that optimize protocol execution over time.

This runtime—the Semantic Vessel, as specified in companion documents—is the target architecture toward which the chat-mode protocol serves as both proof-of-concept and practitioner training ground.

7.3 Appropriate Use of the Chat-Mode Protocol

The chat-mode protocol is not a degraded substitute for the full system. It serves three legitimate and important functions:

- **Proof of structure.** It demonstrates that the protocol's analytical logic produces qualitatively different outputs than single-pass summarization, validating the approach before infrastructure investment.
- **Practitioner training.** It teaches the feel of question-driven reading—the experience of forming an attention field, assembling oriented context, and recognizing incompleteness markers. This embodied experience is prerequisite knowledge for designing and evaluating automated implementations.
- **Immediate utility.** For individual practitioners working with well-documented texts, the chat-mode protocol produces analytical artifacts of genuine professional value: structured semantic maps with explicit boundaries, connections, and incompleteness markers that are unavailable through conventional summarization.

The appropriate framing is therefore: the chat-mode protocol is the **seed form** of the full system—the minimal viable expression through which the architecture proves itself at the scale of one practitioner and one fragment, before it generalizes to document-scale, persistent, governed semantic execution.

References

CoAgnes Framework. (2026). Semantic Vessel: Stepwise Semantic Execution Runtime with Integrated Ethical Governance. Draft Specification v0.1. Document ID: SV-SPEC-2026-001.

CoAgnes Framework. (2026). Semantic Kernel Protocol: Worked Example on GDPR Recital 29. Internal Protocol Document.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

de Bono, E. (1985). Six Thinking Hats. Little, Brown and Company.

End of Document — SKP-GUIDE-2026-001 — v0.1